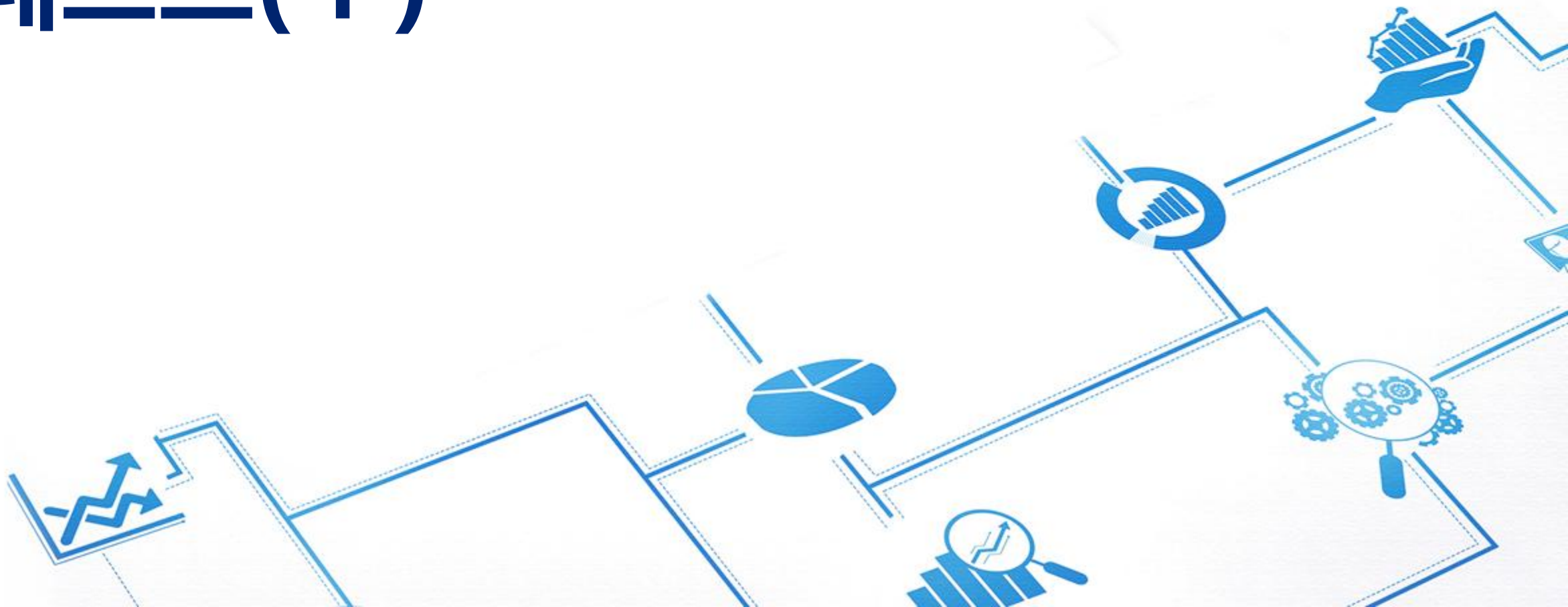


테스트(I)



학습개요

본 주차의 학습목표와 학습내용을 확인하세요.

학습목표

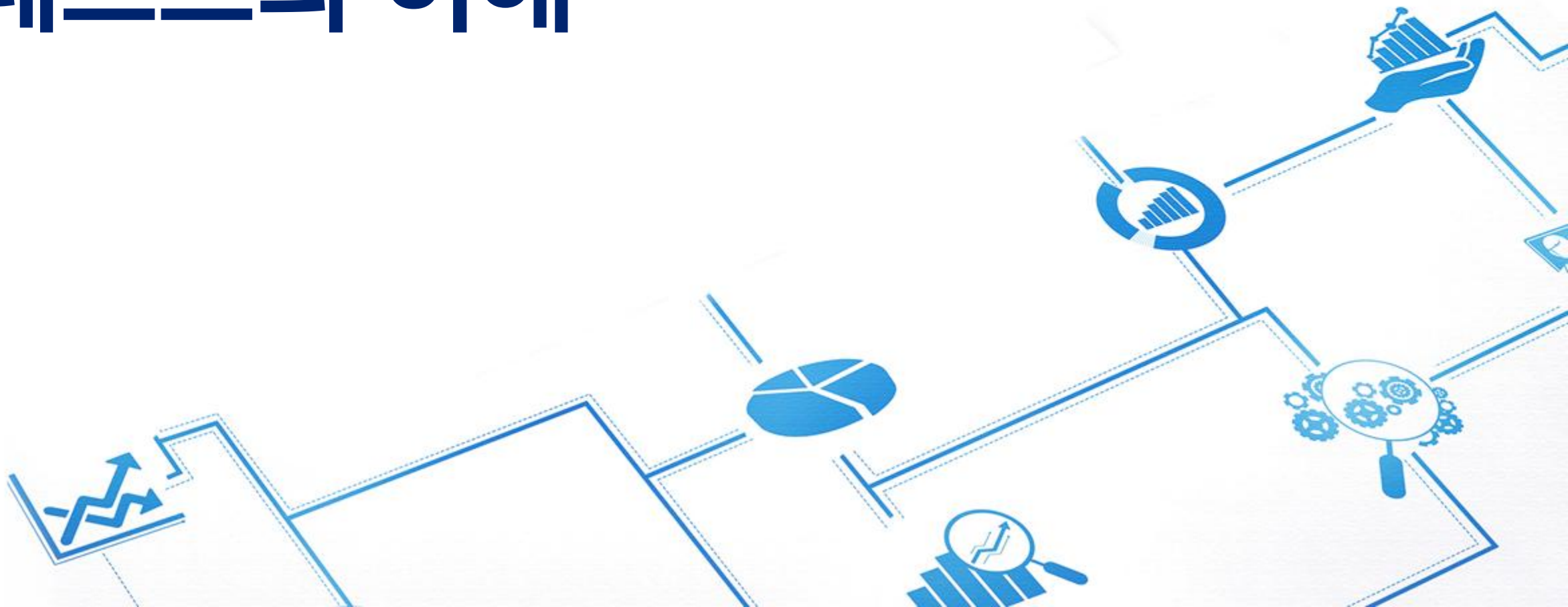
- ☑ 소프트웨어 테스트 프로세스에 대해 알아본다.
- ☑ 시각에 따른 테스트 기법에 대해 살펴본다.
- ☑ 목적에 따른 기법에 대해 살펴본다.
- ☑ 프로그램 실행 여부에 따른 테스트 기법에 대해 살펴본다.
- ☑ 개발 단계에 따른 테스트 기법에 대해 살펴본다.

학습내용

- 1 테스트의 이해, 테스트 분류
- 2 정적 테스트
- 3 동적 테스트
- 4 소프트웨어 개발 단계에 따른 테스트



테스트의 이해



테스트의 이해

테스트와 소프트웨어 테스트

- 소프트웨어 개발에서의 테스트
 - 사후약방문 (死後藥方文): 사람이 죽은 뒤에 약을 짓는다 -> 일을 그르친 뒤에는 아무리 뉘우쳐봐야 이미 늦었다
 - 높은 곳에서 사람이 떨어질 때를 대비해 구급차를 준비하는 것보다 사전에 울타리를 쳐 놓는 것이 훨씬 안전
 - 소프트웨어 개발 과정에서 테스트는 울타리에 비유

테스트의 이해

테스트와 소프트웨어 테스트



死.後.藥.方.文

사람이 죽은 뒤에 약을 짓는다

그림 8-1 테스트가 주는 교훈 : 절벽 밑에 응급차를 마련해놓는 것보다 절벽 위에 울타리를 쳐놓는 것이 안전하다.

테스트의 이해

테스트와 소프트웨어 테스트

- 소프트웨어 오류로 인한 사고 사례
 - 걸프전 당시 페트리어트 미사일 실패
 - 미사일이 1km를 날아갈 때마다 0.5~1초 정도의 아주 미세한 오차 발생 때문
- 방사선 치료기 테락-25의 사고
 - 테락-20을 테락-25로 버전 업할 때, 사고가 없던 테락-20만 믿고 테락-25의 전체 코드에 대한 테스트를 생략했기 때문
- 상업용 우주선 아리안 5호의 공중 폭발
 - 64비트 값을 16비트 정수로 변환하는 과정에서 오버플로가 발생
 - 아리안 4호 프로젝트에서 내버려둔 불필요한 데드 코드를 간과한 결과

테스트의 이해

테스트의 필요성과 특징

- 전문가들이 정의한 소프트웨어 테스트
 - IEEE
 - 시스템이 명시된 요구를 잘 만족하는지, 즉 예상된 결과와 실제 결과가 어떤 차이를 보이는지 수동이나 자동으로 검사하고 평가하는 작업을 의미
 - 조하 만나
 - 테스트는 시스템의 명세까지 완벽하게 옳다고 확신할 수 없고, 테스트 시스템 그 자체가 맞다고 증명할 수 없기 때문에 프로그램을 완전히 테스트할 수 없음
 - 그 밖의 정의
 - 달, 다익스트라, 호어: 테스트는 결함이 있음을 보여줄 뿐, 결함이 없음을 증명할 수는 없음

테스트의 필요성과 특징

- 전문가들이 정의한 소프트웨어 테스트

- 소프트웨어 테스트의 정의

- 소프트웨어에 내에 존재하지만 드러나지 않고 숨어 있는 오류를 발견할 목적으로, 개발 과정에서 생성되는 문서나
 - 프로그램에 있는 오류를 여러 기술을 이용해 검출하는 작업
 - 오류를 찾아내 정상적으로 실행될 수 있도록 하는 정도이지, 소프트웨어에 오류가 없음을 확인시켜주지는 못함
 - 테스트는 오류를 찾고 올바르게 수정하여 프로그램을 작동시킬 수는 있지만, 그 프로그램이 완전하고 정확하다고
 - 증명할 수는 없음

테스트의 필요성과 특징

- 소프트웨어 테스트의 목표
 - 작은 의미
 - 원시 코드 속에 남아 있는 오류를 발견하는 것
 - 결함이 생기지 않도록 예방하는 것
 - 큰 의미
 - 개발된 소프트웨어가 고객의 요구를 만족시키는지 확인시켜주는 것
 - 개발자와 고객에게 사용하기에 충분한 소프트웨어임을 보여주는 것
 - 결과적으로 테스트의 목표는 '개발된 소프트웨어에 신뢰성을 높여주는 것'

테스트의 필요성과 특징

- 소프트웨어 테스트 수행의 어려움
 - 테스트 케이스가 적어 효과에 한계가 있음
 - 완벽한 테스트 케이스를 도출하기 어려움
 - 테스트를 위한 실제 사용 환경을 구축하기 어려움
 - 작은 실수를 발견하기 어려움
 - 테스트 중요성에 대한 인식이 부족함
 - 고객의 요구 사항을 충족시켜야 함
 - 테스트 단계에서만 수행되는 단순한 활동이 아니라 개발 단계와 함께함
 - 파레토 원리를 적용할 수 있음
 - 모듈 단위를 점점 확대해나가며 진행함
 - 완벽한 테스트는 불가능
 - 개발자와 다른 별도의 팀에서 수행
 - 살충제 패러독스(테스트 내성) 문제 해결을 위해 테스트 케이스 업데이트가 필요함

테스트의 이해

테스트의 필요성과 특징

- 소프트웨어 테스트 결함 관련 용어
 - 오류
 - 소프트웨어 개발자에 의해 만들어지는 실수로 결함의 원인
 - 결함
 - 오류에 의해 프로그램이 완전치 못한 것으로, 고장의 원인
 - 주로 프로그램에 필요 없는 정보를 포함하거나 필요한 정보가 없는 경우
 - 결함으로 인해 프로그램이 실행 중에 멈추거나 예외 처리 모듈이 작동될 수 있고 시스템이 작동 불능 상태에 빠질 수 있음
 - 고장, 실패, 문제, 장애
 - 시스템이 요구 사항대로 작동하지 않는 것
 - 요구분석명세서가 잘못되었거나, 이 명세서에 요구사항이 충분히 반영되지 않을 때
 - 기술적으로 불가능한 요구사항이 포함된 경우와 같은 결함에 의해 발생
 - 모든 결함이 반드시 실패를 유발하지는 않음

테스트 절차

- 테스트 절차

- 측정 도구나 장비를 이용해 구현된 소프트웨어가 사용자의 요구를 만족하는지를 테스트 하는 절차
- 소프트웨어 개발 단계와 연계해 구성

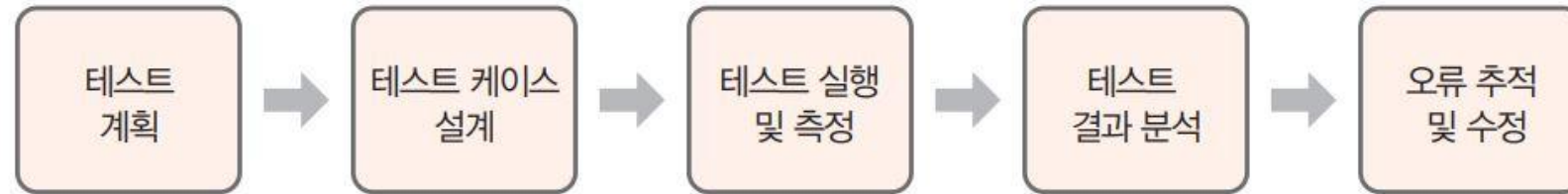


그림 9-2 테스트 절차

테스트 절차

- 테스트 계획

- 테스트 목표를 정의하고, 테스트 대상 및 범위를 결정하며, 테스트 계획서를 작성하고 검토

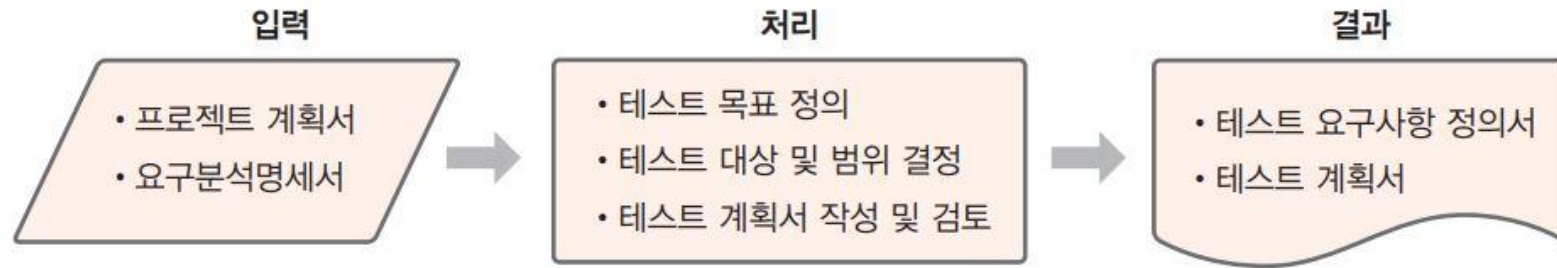


그림 9-3 소프트웨어 테스트 계획 단계

테스트 절차

- **테스트 목표 정의**
 - 요구분석명세서를 기반으로 테스트할 목표를 정의
 - 테스트 항목 중에서 어떤 항목을 중점적으로 테스트할 것인지 명확히 나타냄 (예: 보안성과 안정성 중 무엇을 중점적으로)
- **테스트 대상 및 범위 결정**
 - 테스트할 대상과 범위를 결정하는데, 이때는 업무 시스템별로 구분할 수 있음
- **테스트 계획서 작성 및 검토**
 - 테스트의 목적, 담당 인원, 테스트 전략과 접근 방법 수립, 필요한 자원 및 자원 확보 일정, 실시할 테스트의 종류,
 - 적용할 테스트 기법, 일정 등에 관한 정보를 기록
 - 작성된 테스트 계획서를 가다듬고, 이를 확정

테스트 절차

- 테스트 계획

- 테스트 설계 기법을 정의하고, 도출된 테스트 케이스에 입력 값으로 사용 할 원시 데이터를 작성

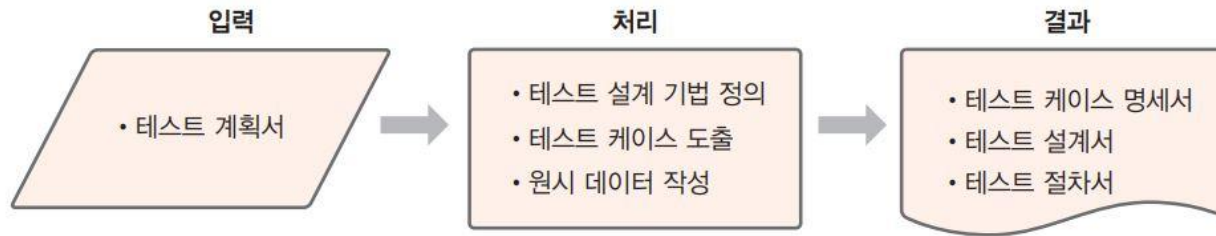


그림 9-4 소프트웨어 테스트 케이스 설계 단계

- 테스트 케이스 설계 기법 정의

- 테스트할 프로젝트 문제의 성격을 파악하고 이 문제에 적합한 테스트 기법을 선정

- 테스트 케이스 도출

- 테스트 기법이 결정되면 이 기법을 이용해 테스트 케이스를 도출

- 원시 데이터 작성

- 결정된 테스트 케이스를 수행하기 위해 필요한 원시 데이터를 작성

테스트 절차

- 테스트 실행 및 측정

- 테스트 환경을 구축하고 도출된 테스트 케이스를 이용해 테스트를 실시하고 테스트 실행 결과를 문서화

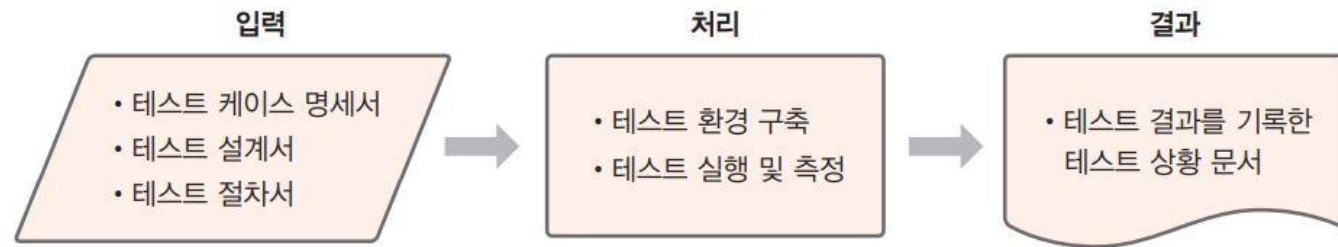


그림 9-5 소프트웨어 테스트 실행 및 측정 단계

- 테스트 환경 구축

- 테스트 계획서에 정의된 환경 및 자원을 설정해 테스트를 실행할 준비를 함

- 테스트 실행 및 측정

- 정의된 테스트 케이스를 실행하고 실행 결과를 측정

테스트 절차

- 오류 추적 및 수정

- 테스트 결과 어디에서, 어떤 종류의 오류가 발생했는지 확인하고 수정

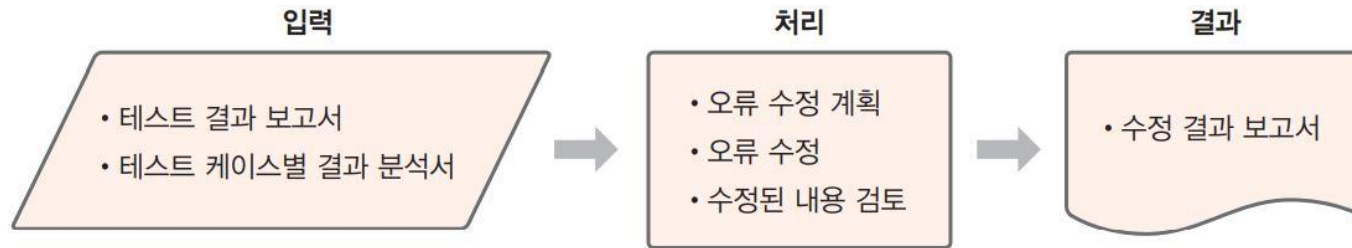


그림 9-7 오류 추적 및 수정 단계

- 오류 수정 계획

- 테스트 결과 보고서를 기반으로 오류가 발생한 위치를 찾아냄
- 오류 수정 우선순위를 결정해 오류 제거 계획을 수립

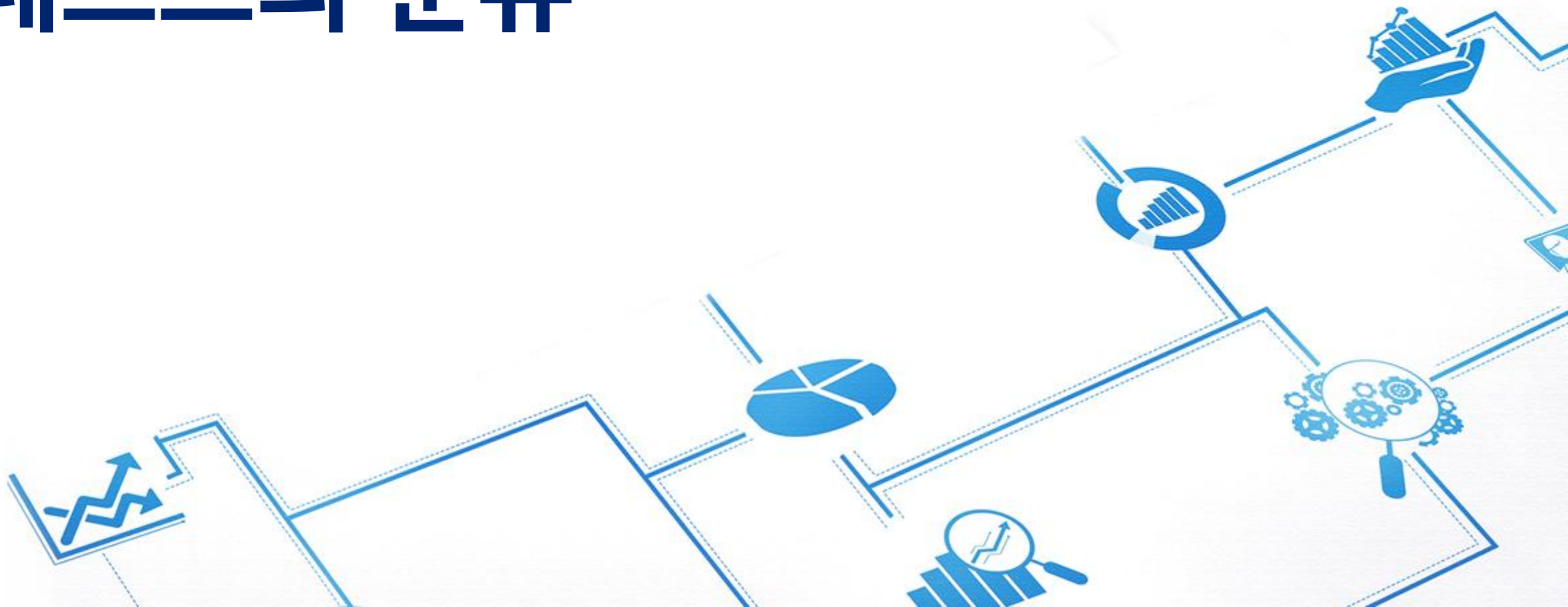
- 오류 수정

- 오류 제거 계획을 기초로 디버깅 도구 등을 이용해 오류를 수정

- 수정된 내용 검토

- 수정된 코드를 검토한 후 오류 수정 결과 보고서를 작성

테스트의 분류



시각에 따른 테스트

- **확인 테스트**

- 잘못 개발된 연산 프로그램의 예시

- 사용자가 1부터 10까지 곱하는 프로그램을 주문했는데, 개발자가 착각해 1부터 10까지 더하는 프로그램을 개발 할 경우
 - 확인 테스트를 수행하면 1부터 10까지 더하는 계산과정이 정확하고, 결과가 55가 맞는지 체크
 - 하지만 사용자가 원하는 것은 덧셈이 아닌 곱셈이므로 확인 테스트만으로는 사용자가 원하는 것을 개발했는지 알 수 없음

- **확인 테스트의 개요**

- 각 단계에서 개발자의 시각으로 테스트하는 것으로, 설계서대로 만들었는지를 테스트
 - 이전 단계에서 생성된 산출물이 현 단계의 산출물에 정확히 반영되었는지를 테스트
 - 요구분석명세서의 내용이 설계서에 전부 반영되어 있어야 하고, 설계서의 내용대로 코딩이 이루어져야 함

테스트의 이해

시각에 따른 테스트

- 확인 테스트

- 확인 테스트의 한계

- 설계서가 요구분석명세서를 완벽하게 반영하지 않을 경우에는 결국 틀린(또는 부족한) 요구사항을 가지고 개발
 - 개발 과정을 완벽하게 테스트했다고 해도 사용자에게는 쓸모없거나 불완전한 개발 결과가 나오게 됨

- 검증 테스트

- 앞의 예시에서 검증테스트를 실시할 경우 곱셈을 했는지 체크, 즉 사용자의 요구사항대로 만들었는지를 테스트
 - 사용자의 목적에 맞게 개발했는지는 요구분석명세서대로 만들었는지가 기준
 - 확인 테스트와 검증 테스트를 함께 사용해야 좀 더 강력한 테스트가 될 수 있으며, 이 둘을 묶어 V & V

테스트의 이해

사용 목적에 따른 테스트

- 목적에 따른 분류

- 성능 테스트

- 소프트웨어의 효율성을 진단하는 테스트
 - 사용자의 요구사항 중에서 성능과 관련된 요구사항을 시스템이 얼마나 준수하는지 테스트
 - 시스템의 전 후 관계에서 소프트웨어 실행 시간을 테스트
 - 일반적으로 예상된 부하에 대한 실행 시간, 응답 시간, 처리 능력 등을 체크하고, 자원 사용량 등을 테스트

- 스트레스 테스트

- 평소보다 많은 비정상적인 값, 양, 빈도, 부피 등으로 부하를 발생시켜 부하가 최고치인 상황에서 시스템의 반응을 살피고 이때 발생하는 오류를 찾는 것
 - 비정상적이고 과도한 부하 상황에서 시스템이 잘 견디는지, 발생하는 오류에는 어떤 것들이 있는지를 확인

테스트의 이해

사용 목적에 따른 테스트

- 목적에 따른 분류
 - 보안 테스트
 - 부당하고 불법적인 침입을 시도해, 시스템을 보호하기 위해 구축된 보안 시스템이 불법적인 침투를 잘 막아내는지 테스트
 - 외부 해커의 입장에서 비밀번호를 알려고 시도하거나, 구축해 놓은 방어 체계를 뚫거나 파괴하는 등 여러 가지 방법을 사용

테스트의 이해

사용 목적에 따른 테스트

- 목적에 따른 분류

- 안정성 테스트

- 며칠 동안 부하를 주면서 시스템이 안정적으로 돌아가는지를 테스트
 - 메모리 누수: 소프트웨어 프로그램은 메모리가 필요할 때 요청해 사용하는데 사용 후 메모리를 반납하지 않아 사용 가능한 메모리량이 점점 줄어드는 현상

- 복원 가능성 테스트

- 소프트웨어를 고장 나게 해 놓고 소프트웨어 복구가 잘 되는지 확인해보는 테스트
 - 만약 소프트웨어에서 자동으로 회복이 이루어진다면 소프트웨어 회복의 완벽성을 평가
 - 기술자에 의해 소프트웨어가 복구된다면 복원 가능성 테스트에서 검사
 - 복원 가능성 테스트는 운영체제, DBMS, 통신용 소프트웨어의 안정성 테스트에 적용 가능

테스트의 이해

사용 목적에 따른 테스트

- 프로그램 실행 여부에 따른 테스트

- 정적 테스트

- 자동차 수리시 보닛을 열어 부품 상태는 괜찮은지, 나사가 잘 조여 있는지 등을 면밀히 살펴보고 고장 난 곳을 찾음
 - 정적 테스트: [그림 9-8]처럼 프로그램을 실행하지 않고 코드를 검토하며 오류를 찾는 방법



그림 9-8 자동차 내부 상태를 체크해 고장을 찾는 방법

테스트의 이해

사용 목적에 따른 테스트

- 프로그램 실행 여부에 따른 테스트

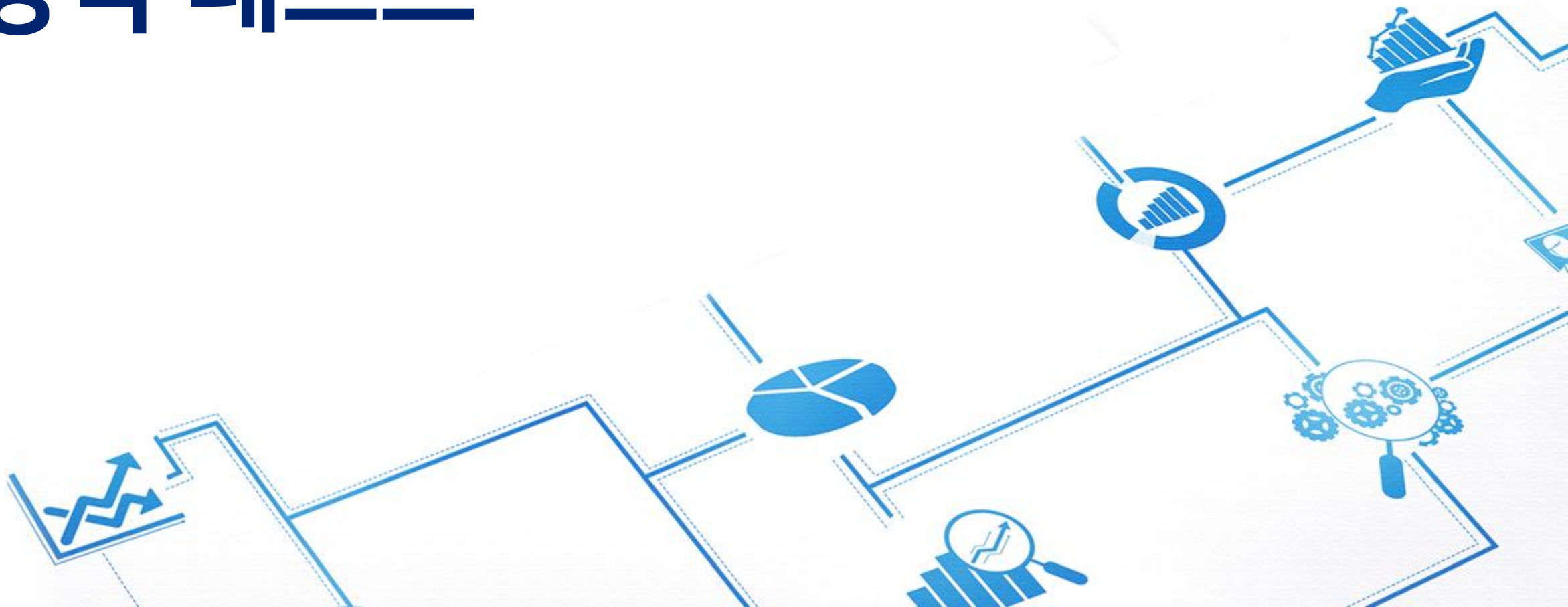
- 동적 테스트

- 직접 주행하면서 엔진 소리가 필요 이상으로 크지는 않은지, RPM 수가 정상 수치보다 높게 올라가는지 확인
 - 핸들의 떨림 현상이나 브레이크를 밟았을 때 밀리는 정도를 체크해 고장의 원인을 찾음
 - 동적 테스트: [그림 9-9]처럼 프로그램을 실행하면서 찾는 경우



그림 9-9 자동차 주행을 통해 고장을 찾는 방법

정적 테스트



정적 테스트

- 정적 테스트의 개요

- 프로그램 코드를 실행하지 않고 여러 참여자가 모여 소프트웨어 개발 중에 생성되는 모든 명세나 코드를 검토해서
- 실패보다는 결함을 찾아내는 방법
- 정적 테스트를 위해 도구를 사용하면 프로그램 코드 분석뿐만 아니라 HTML, XML과 같은 산출물도 검토할 수 있음

정적 테스트

• 정적 테스트의 개요



그림 9-10 정적 테스트 방법

정적 테스트

- 비공식 검토

- 소프트웨어 개발 과정에서 생성되는 문서, 프로그램 과 같은 산출물을 동료와 함께 책상에서 검사하는 것
- 제품을 검토할 목적으로 하는 간단한 만남 등이 이에 속함
- 개별 검토와 동료 검토가 주로 해당

- 공식 검토

- 동료와 소프트웨어 기술 전문가들이 수행하는 소프트웨어 품질 제어 활동
- 결함을 찾기 위해 정의된 절차에 따라 적절히 계획되고 통제
- 검토 회의와 소프트웨어 검사가 주로 이런 방식으로 수행.

정적 테스트

- 검토 내용
 - 원시 코드상에 존재하는 오류를 검토
 - 소프트웨어가 사용자의 요구를 충분히 반영했는지 검토
 - 소프트웨어가 미리 정의된 표준을 지키는지 검토
 - 소프트웨어 개발 방식이 일관적인지 검토
 - 소프트웨어가 관리하기 쉬운지 검토

정적 테스트

- 공식 검토

- 공식 검토 수행 절차



그림 9-11 공식 검토 절차

- ① 계획: 참가 인원을 몇 명 정도로 할 것인지, 누구를 참여시킬지, 참가자 개개인에게 어떤 역할을 맡길 것인지 고민하고 결정
- ② 착수: 모든 준비가 끝났으면 참석자들에게 회의 문서를 배포하고 본 검토회의의 취지와 목표, 절차 등에 관해 설명
- ③ 개별 준비: 본회의를 하기 전에 체크리스트를 사용해 참석자들에게 미리 한 번 검토하도록 함
- ④ 검토회의 수행: 본회의에서 참석자들은 발견한 결함 내용과 결함의 처리 방법 등을 정리하고 문서로 작성
- ⑤ 재작업 및 수정: 문서 작성자가 회의에서 정리한 결함을 담당 개발자에게 전달하고 개발자는 수정작업 실시
- ⑥ 완료 작업, 후속 처리 확인: 정리된 결함이 조치되었는지 확인, 공식적인 검토에서는 관련 측정치 수집, 검토 종료 기준 체크

정적 테스트

- 개별 검토

- 체크리스트를 가지고 본인이 개발한 코드와 산출물 등을 검토하는 것
- 본인 스스로 검토하기 때문에 여러 방법 중 가장 간단한 방법이지만, 상대적으로 객관성이 떨어짐

- 동료 검토

- 동료에게 원시 코드나 여러 가지 산출물에 대한 검토를 의뢰하여 오류를 찾는 방법
- 정해진 형식도 없고 별도의 격식을 차린 회의를 수행할 필요가 없어 비공식 검토에 속함
- 동적 테스트에서 발견하기 어려운 산출물에서 발생할 수 있는 누락과 같은 결함을 조기에 발견할 수 있는 이점

정적 테스트



그림 9-12 훈수 두는 모습

정적 테스트

- 검토회의

- 개발자가 소집한 전문가들에 의해 개발자의 작업을 검토
- 3~5명 정도의 전문가(프로젝트 팀장, 다른 개발자, 품질 보증단장)들이 절차에 따라 평가
- 설계 문서들이 고객의 요구 사항을 정확히 명시하고 있는지 여부, 작업 진척 상황 등 확인

- 검토회의의 준비 및 주의 사항

- 검토회의를 통해 검토받고자 하는 개발자는 회의 자료(검토받을 문서 또는 원시 코드)를 준비하고 회의 일정을 계획
- 검토회의의 결과를 인사 평가 자료로 사용해서는 안됨
- 회의 자료는 회의가 있기 4~6일 전에 전달해 회의 참여자가 미리 살펴보고 올 수 있도록 함
- 검토회의는 문제점을 찾는 데 주안점을 두고, 그 문제의 해결은 검토회의 이후로 미룸
- 검토회의 때 발견해 작성한 오류 리스트는 회의가 끝난 후 개발자에게 전달
- 검토회의 시간은 너무 길지 않아야 함, 일반적으로 1~2시간 이내

소프트웨어 검사

- 소프트웨어 검사의 개요
 - 발견된 문제점 수정 지침까지 제시 및 수정을 잘하고 있는지 추후에 조사
 - 원시 코드뿐 아니라 각 단계 산출물의 문서 등을 포함하여 분석하고 품질을 평가
 - 소프트웨어 품질 보증 기법으로 유용
 - 공식 검토에 속함

소프트웨어 검사

• 소프트웨어 검사 절차

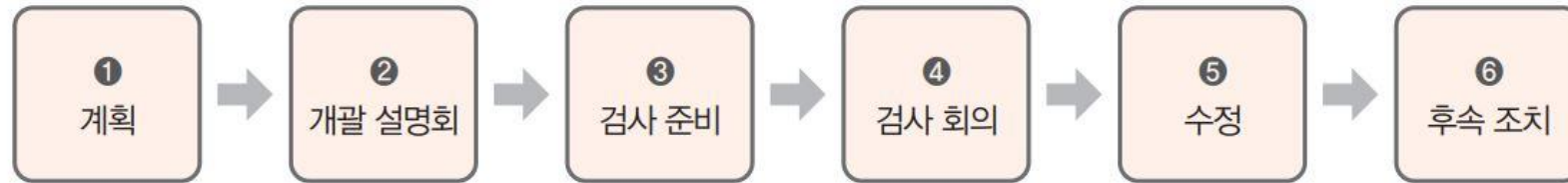


그림 9-13 소프트웨어 검사 절차

표 9-1 검사 참가자 및 역할

참가자	역할 및 책임
주재자 ^{moderator}	검사를 전체적으로 기획, 중재하는 사람
개발자 ^{author}	검사받을 프로그램(또는 문서)을 만든 사람
기록자 ^{recorder}	진행 상황을 기록하는 사람
검토자 ^{reviewer}	• 전달받은 자료를 검토하는 모든 참가자 • 결함을 찾을 뿐 해결 방안은 제시할 필요 없음

소프트웨어 검사

- 소프트웨어 검사를 할 때 지켜야 할 원칙
 - 검사 회의는 2시간 이내로 하는 것이 적당
 - 검사 회의에 참가하는 총 인원은 5명 내외가 적당
 - 검토자가 사전에 읽어보고 올 수 있도록 최소한 2일 전에는 자료를 전달
 - 주제를 벗어나는 개별적인 질문은 삼가함
 - 발견된 오류에 대해서는 반드시 문서화하여 기록으로 남김
 - 검사 회의 목적은 오류를 발견하는 것이다. 따라서 오류 수정은 하지 않음
 - 검사 회의가 끝나면 문서를 정리하여 관련된 사람들에게 전달

소프트웨어 검사

• 설계와 구현시 소프트웨어 검사

표 9-2 설계와 구현 시 소프트웨어 검사

설계 검사	• ‘사용자가 요구한 기능들이 설계에 반영되었는가’와 같은 완전성 검사 • ‘사용되는 용어가 서로 다르게 해석되지 않았는가’와 같은 일관성 검사 • ‘모듈 사이에 인터페이스가 잘 정의되었는가’와 같은 정확성 검사
원시 코드 검사	• ‘프로그램에서 변수를 사용하기 전에 선언이 되었는가’와 같은 데이터 결함 검사 • ‘제어문에서 조건이 정확한가’와 같은 제어 결함 검사 • ‘모든 입력 변수가 사용되고 있는가’와 같은 입출력 결함 검사 • ‘인자의 데이터 타입, 개수, 순서가 일치하는가’와 같은 인터페이스 결함 검사

END

